

⚡ NEXT-GEN ATS INTELLIGENCE

# Resume Intelligence & ATS Studio

An autonomous NLP engine that reads resumes, understands them, and structures every candidate into clean, queryable data.

---

NLP Internship Project • Applicant Tracking System Engine

*Python • spaCy • RapidFuzz • Streamlit*

# The Problem

Recruitment doesn't scale with manual screening



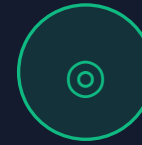
## Volume Overload

A single posting can attract hundreds of resumes — far more than any recruiter can manually read.



## Inconsistent Formats

PDFs, Word docs, two-column layouts, creative templates — every resume structures information differently.



## Lost Signal

Buried in unstructured text, key qualifications get missed or take too long to surface for filtering.

*Goal: automatically read, understand, and structure resumes into standardized data  
recruiters can filter and query instantly.*

# The Solution

A deterministic, modular resume parsing engine

The ResumeParser engine ingests any PDF or DOCX resume and runs it through a five-stage pipeline — extraction, cleaning, section segmentation, entity recognition, and skill matching — producing one standardized JSON record per candidate.

- Handles PDF and DOCX formats natively
- Resilient to varied section headers and layouts
- Fail-closed extraction — never fabricates data
- Matches skills against a 75+ term ontology

6

Pipeline  
Modules

75+

Ontology  
Skills

**Deterministic  
Output Schema**

# Pipeline Architecture

Every resume flows through six deterministic stages



*Stages 4–5 run across three parallel extractors: `contact_extractor`, `ner_extractor`, and `skill_matcher` — each independently testable, then merged by `resume_parser.py` into one coordinated record.*

## Design principle: fail-closed extraction

If a field can't be confidently extracted, the parser leaves it empty rather than guessing — preventing a job description bullet from becoming a fake company name, or a date range from becoming a phantom job entry.

# Tech Stack

Purpose-built tools for each stage of the pipeline



**Python 3.10+**

Core language



**pdfminer.six**

PDF text extraction



**python-docx**

DOCX text extraction



**spaCy (en\_core\_web\_sm)**

NER + custom EntityRuler



**RapidFuzz**

Fuzzy skill matching



**Regex (re)**

Contact & pattern extraction



**Pandas**

Data handling utilities

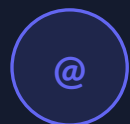


**Streamlit**

Interactive dashboard UI

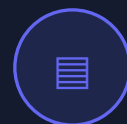
# Core Features

What the engine actually catches that naive parsing misses



## Robust Contact Extraction

RFC 5322 email matching, international phone normalization, LinkedIn & GitHub/portfolio detection — with a strict boundary so experience bullets never leak into contact fields.



## Boundary-Safe Segmentation

Recognizes section headers across ALL CAPS, Title Case, colon-suffixed, and markdown styles — confining each section's content correctly.



## Entity-Aware Parsing

Custom spaCy EntityRuler patterns detect degree titles, institutions, and job title/company splits — even whitespace-only separators.



## Semantic Skill Matching

Exact + fuzzy (RapidFuzz) matching against a 75+ skill ontology resolves aliases like 'React.js' → 'React' and 'K8s' → 'Kubernetes'.

# Skill Ontology & Matching Engine

A structured taxonomy, not a flat keyword list

Every skill is stored with a canonical name, category, and alias list — so 'Postgres', 'React.js', and 'K8s' all resolve to one clean, consistent label in the output.

```
{ "canonical": "React",  
  "category": "Web & Frontend",  
  "aliases": ["React.js", "ReactJS", "React Native"] }
```

## Matching strategy:

- Exact, word-boundary-safe regex for symbol-heavy terms (C++, .NET, CI/CD)
- RapidFuzz token\_sort\_ratio for phrasing variants, threshold-gated (~85)
- Length guards to prevent short/generic tokens from over-matching

# 75+

Canonical Skills Indexed

- Languages
- Web & Frontend
- Backend Frameworks
- Databases
- Cloud & DevOps
- Data Science & AI
- Big Data
- Methodologies

# Live Dashboard — Document Ingestion

A polished Streamlit front-end, not a bare upload form

>>

Deploy

⚡ NEXT-GEN ATS INTELLIGENCE

Enterprise Edition • v2.4

## Resume Intelligence & ATS Studio

Autonomous document ingestion, semantic skill graph extraction, and precision role alignment dashboard.

> ⚡ ATS Engine Configuration & Matching Sensitivity

📁 Ingestion & Document Source

Choose an ingestion method below. Upload a candidate document, select a benchmark resume, or paste raw text.

☒ 📁 Upload File (PDF / DOCX)

☐ 📁 Load Pre-Built Sample

☐ 📄 Direct Text Stream

Upload Candidate Document (PDF or DOCX)

📄 New-Yor...eative.pdf

70.0KB

ⓧ

+

?

Resume Intelligence & ATS Studio

8



# Live Dashboard — Parsed Candidate Profile

Structured data rendered as a recruiter-ready profile card

>>

⚡ ATS BENCHMARK SCORE

ATS Match Fit: 95%

🔥 Profile Completeness: 9 technical & domain skills verified, 4 career milestones, 3 education records.

95%

MATCH FIT

Deploy

RC

Robert Cooper

Security Guard

Safety conscious, attentive Security Guard with eight years experience in protecting and guarding VIP individuals in the military and defense sectors. Proficient at observing surroundings and immediate settings for possible threats of nonhuman and human nature. Presently finishing a bachelor's in criminal justice and qualified as a CPO (Certified Protection Guard).

✉ email@email.com

☎ 3868683442

🔗 Portfolio

SKILLS MATCHED

⚡

9 Verified

EXPERIENCE

👛

4 Milestones

EDUCATION

🎓

3 Credentials

PROJECTS

🚀

0 Showcases

📄 Overview & Timeline

🎯 Target Job Matcher

📄 Styled Resume Preview

📊 Skill Analytics

🔍 ATS JSON Inspector

📄 Raw Document Text

# Standard ATS JSON Output

Every resume becomes one clean, predictable record

```
{
  "contact": {
    "name": "Alexander Hayes",
    "email": "alex.hayes@email.com",
    "phone": "+1 (415) 555-0192",
    "linkedin": "linkedin.com/in/alex-hayes"
  },
  "experience": [
    { "company": "Stripe",
      "title": "Lead Software Engineer",
      "dates": "Jan 2021 - Present" }
  ],
  "education": [ { ... } ],
  "skills": ["Python", "AWS", "Docker", ...],
  "projects": [ { ... } ]
}
```

## contact

name, email, phone, linkedin, portfolio

## summary

professional summary text

## experience

company, title, dates, description

## education

institution, degree, dates

## skills

canonicalized skill list

## projects

name, description

# Testing, Challenges & Learnings

Real resumes surface edge cases sample data never does

## Validation approach

- Unit tests for contact extraction and skill matching
- Batch verification across multiple sample resumes
- Manual testing on real, non-generated resumes
- Direct pipeline testing bypassing the UI, to isolate bugs

## Key issues found & resolved

- Whitespace-only title/company separators silently failed — added a dedicated pattern
- Continuation bullets were spawning fake job entries — now merged into descriptions
- Standalone date lines fragmented education records — now attached to the parent entry
- Near-duplicate skill phrasings appeared twice — ontology aliasing deduplicated

*Takeaway: unconventional PDF layouts with irregular bullet structures remain a known limitation — an honest edge case, not a hidden flaw, and a clear direction for future hardening.*

# Future Scope

Where this engine goes next

## FastAPI Backend

A production REST endpoint for programmatic, non-UI resume ingestion.

## Fine-tuned NER Model

Train on labeled resume data for higher accuracy on unconventional layouts.

## Job-Fit Scoring

Expand the Target Job Matcher into a full ranking model across a candidate pool.

---

# Thank You

Resume Intelligence & ATS Studio — built end-to-end as an NLP internship project.